

손찬양 백엔드 엔지니어링 포트폴리오

분산 상태 수렴, 동시성 제어, 접근 권한, 데이터 재처리와 AI 요청 정책을 구현하고 테스트로 검증했습니다.

트랜잭션 · 권한 · 동시성

상태 수렴과 업무 불변식 보호

LLM · 데이터 · 개인화

정책 파이프라인과 데이터 품질 추적

01 StockRush

재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남

04 AI Gateway

애플리케이션마다 중복되는 LLM 정책과 외부 모델 장애 처리

03 Member Event Consistency

중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액

05 CDC Data Platform

중복 CDC, 복구 이력 손실과 자동 연결되지 않은 구간의 중단 전달

02 Enterprise Policy RAG

비인가 문서 노출과 근거 없는 답변 생성

06 Fashion Personalization Platform

중복·실패 이벤트가 프로필과 추천 스냅샷을 오염함

01 StockRush

분산 커머스 / 주문 상태 수렴 / 이벤트 기반 백엔드

- 문제** 재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남
- 설계** Saga 상태 전이, 트랜잭션 Outbox, 멍든 소비, 종료 상태 SQL 조건
- 결과** 동일 이벤트 2회에도 Outbox 1건, 늦은 결제 승인 무시, 발행 5회 실패 뒤 FAILED

04 AI Gateway

LLM Gateway / 라우팅 / 비용 보호 / 캐시 / 폴백 / 관측

- 문제** 애플리케이션마다 중복되는 LLM 정책과 외부 모델 장애 처리
- 설계** API 키·할당량, 2단계 캐시, 정책 라우팅, 제한된 재시도·대체 경로와 요청 기록
- 결과** 8단계 처리 순서, 4개 실행 방식과 4개 라우팅 전략을 회귀 테스트로 고정

03 Member Event Consistency

동시성 / 업무 불변식 / DB 보호 조건 / Redis-RabbitMQ 비교

- 문제** 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액
- 설계** DB 제약·조건부 갱신을 최종 보호 조건으로 두고 Redis-RabbitMQ를 경합별 비교
- 결과** 동시 8건→보상 1건, 60원 차감 2건→잔액 40, 용량 3→정확히 3건 발급

05 CDC Data Platform

CDC / Kafka / Lakehouse / Lineage / Replay

- 문제** 중복 CDC, 복구 이력 손실과 자동 연결되지 않은 구간의 중단 전달
- 설계** 독립 실행 환경·제어 API·레이크하우스 실험, 이벤트 ID, 처리 장부와 재처리 상태
- 결과** 원본 7건 중 6건 반영·중복 1건 억제, 4개 원천 테이블과 분석 마트의 계보 추적

02 Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

- 문제** 비인가 문서 노출과 근거 없는 답변 생성
- 설계** 검색 전 권한 필터, 인용 범위 재검사, 근거 없는 답변 거절, 질의 로그
- 결과** 문서 5건 중 허용 3건만 검색, 근거가 없으면 답변과 인용을 비운 거절 응답

06 Fashion Personalization Platform

Python API / 커머스 개인화 / 추천 배치 / 이벤트 처리

- 문제** 중복·실패 이벤트가 프로필과 추천 스냅샷을 오염함
- 설계** 멍든 이벤트 처리, 재시도·DLQ, 규칙 기반 추천 순위와 배치 스냅샷
- 결과** 동일 키 1회 반영, 3회 실패 뒤 DLQ, 상품 6개 추천 순위와 스냅샷 2개 이상·지연 0건

역량을 보여주는 구현 근거

상태·불변식 모델링 복잡한 동시 처리 문제를 상태 전이와 업무 불변식으로 분해하고 경계마다 보호 장치를 선택했습니다.	코드 근거	OrderSagaEventHandler · SqlPointSpendRepository
	구현 결과	주문 Saga와 Outbox로 서비스 간 상태 수렴 · DB 고유·검사 제약과 조건부 갱신으로 최종 불변식 보호
	프로젝트	StockRush · Member Event Consistency
실패 복구 설계 실패를 숨기지 않고 재시도, DLQ, 재처리와 대체 경로처럼 추적하고 다시 처리할 수 있는 상태로 남겼습니다.	코드 근거	OutboxRelayService · FallbackChain · ReplayRequestService
	구현 결과	Kafka·Outbox 중단 뒤 재발행과 먹등 재처리 · LLM 대체 경로, CDC 재처리와 이벤트 재등록 경로
	프로젝트	StockRush · AI Gateway · CDC Data Platform · Fashion Personalization
보안·정책 경계 인증, 권한, 할당량과 외부 모델 정책을 도메인 로직 밖의 명시적인 진입 경계에서 적용했습니다.	코드 근거	SecurityConfig · RetrievalService · GatewayPipeline
	구현 결과	OIDC/JWT Gateway와 리소스 소유권 검사 · 검색 전 권한 필터와 LLM 요청 정책
	프로젝트	StockRush · Enterprise Policy RAG · AI Gateway
데이터·AI 파이프라인 RAG 답변, CDC 마트와 추천 스냅샷에서 입력과 결과를 잇는 식별자와 변환 메타데이터를 보존했습니다.	코드 근거	AnswerService · CanonicalIngestService · rank_products
	구현 결과	인용·거절을 포함한 RAG 응답 흐름 · CDC 데이터 계보와 행동 기반 추천 스냅샷
	프로젝트	Enterprise Policy RAG · CDC Data Platform · Fashion Personalization
관측·운영 제어 요청 기록, 처리 지연, 비용, DLQ와 최신성을 API와 화면에 노출해 장애와 품질 상태를 추적했습니다.	코드 근거	QueryLogRepository · UsageRollupAggregator · PipelineQualityService
	구현 결과	LLM 지연 시간·토큰·비용과 요청 기록 · 커넥터 지연·품질 기준·배치 최신성 리포트
	프로젝트	Enterprise Policy RAG · AI Gateway · CDC Data Platform · Fashion Personalization
재현 가능한 검증 동시성, 중복, 장애와 정적 데모를 반복 실행 가능한 스모크·통합·회귀 테스트로 고정했습니다.	코드 근거	MvpLiveInfrastructureIT · verify-static-demo · run_portfolio_audit
	구현 결과	Docker Compose와 Testcontainers 인프라 검증 · pytest, 오프라인 회귀 테스트와 감사 실행 자동화
	프로젝트	StockRush · Member Event Consistency · AI Gateway · CDC Data Platform

StockRush

분산 커머스 / 주문 상태
수령 / 이벤트 기반 백엔드

담당 범위

개인 프로젝트 / 설계,
구현, 검증

주요 기술

Spring Boot

Kafka

PostgreSQL

Keycloak

결제 지연과 이벤트 중복 뒤에도 주문 상태를 수렴시키는 커머스

한정 재고 주문에서 결제 지연·이벤트 중복·Kafka 중단이 겹쳐도 주문을 취소·확정 상태로 수렴시키도록 Saga, Outbox, 멍든 소비자를 구현했습니다.

시스템 흐름

실패 조건과 보호 경계

Client
01Gateway
02Order Saga
03Inventory /
Payment
04Outbox /
Kafka
05Read Model
06

멍든 소비

동일 이벤트 2회 →
Outbox 1건

처리 완료 이벤트 키로 중복 차
단

종료 상태 조건

늦은 승인 → 무시
취소 주문 상태 유지

발행 재시도

5회 실패 → FAILED

실패 상태와 재처리 경로 보존

해결한 문제

재시도 뒤 주문·재고·결제·조회 상태가 서로 어긋남

동시 주문, 결제 실패와 지연, Kafka 중단, 이벤트 재처리 상황에서는 주문 상태와 재고 수량, 결제 결과, 조회 모델이 서로 다른 속도로 바뀝니다. 서비스 경계와 복구 흐름을 코드와 시나리오로 분리해 최종 상태 수렴을 검증했습니다.

핵심 설계 판단

Saga 상태 조율

재고·결제·쿠폰·출고 상태 전이를 주문 서비스에서
조율

신뢰할 수 있는 이벤트 발행

DB 변경과 Outbox 행을 함께 저장해 발행 실패를
재처리

대표 코드 근거

PersistentCreateOrderService.create

파일 services/order-
service/src/main/java/com/stockrush/order/application/Persistent
CreateOrderService.java

```
boolean saved =
orderRepository.saveIfAbsent(result.order(),
command.idempotencyKey());
if (!saved) {
return
CreateOrderResult.replayed(replayOrder(command.id
empotencyKey(), command.memberId()));
}
outboxEventRepository.save(result.outboxEvent());
```

보호 규칙 주문 저장과 Outbox 기록을 같은 트랜잭션에 두고 멍든 충돌은
기존 주문 재조회로 수렴시킵니다.

연결 테스트 services/order-
service/src/test/java/com/stockrush/order/infra/persistence/P
ersistentCreateOrderServiceIntegrationTest.java
PersistentCreateOrderServiceIntegrationTest

구현 결과 ems_and_pending_outbox_event_together

동일 이벤트를 두 번 처리해도 Outbox가 1건만
남고, 취소 뒤 늦게 도착한 결제 승인은 주문 상태를 바꾸지
않았습니다.

검증 결과

- OrderSagaEventHandlerIntegration
동일 이벤트 2회 처리에도 Outbox 1건
남도록 고정합니다.

- 취소 완료 뒤 늦게 도착한 결제 승인이
주문을 되살리지 않는 상태 guard를
검증합니다.

확정한 범위와 남은 경계

코드와 통합 테스트로 멍든 소비·종료 상태 조건·
발행 재시도를 다뤘으며 실패·다중 리전·운영 규모
부하는 포함하지 않았습니다.

Member Event Consistency

동시성 / 업무 불변식 / DB 보호 조건 / Redis-RabbitMQ 비교

담당 범위

2026.05-2026.06 / 개인 프로젝트 / Spring Boot 백엔드, 인프라 비교, 검증 대시보드

주요 기술

Spring Boot

PostgreSQL

Redis

RabbitMQ

보상·쿠폰·포인트 불변식마다 다른 동시성 제어를 적용한 백엔드

첫 로그인 보상·쿠폰 발급·포인트 차감의 불변식을 PostgreSQL에서 최종 보호하고, Redis 잠금과 RabbitMQ 단일 소비자의 적용 차이를 동시 요청으로 비교했습니다.

전략 비교

실패 조건과 보호 경계

 Scenario API 01	 DB Baseline 02	 Redis + DB 03	 RabbitMQ Lane 04	 Outbox 05	 Infra Tests 06
 최초 보상 동시 요청 8건 → 보상 1건 7건 거절 · 중복 0건		 포인트 차감 100 - (2 × 60) → 40 1건 성공 · 음수 잔액 0건		 쿠폰 용량 요청 8건 / 정원 3건 3건 발급 · 5건 거절	

해결한 문제

중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액

모든 회원 이벤트를 하나의 사용자 ID 잠금으로 묶으면 특정 키에 요청이 몰리고, 아무 제어 없이 처리하면 중복 보상, 쿠폰 초과 발급, 포인트 음수 잔액과 후속 이벤트 누락이 발생합니다.

핵심 설계 판단

DB를 최종 보호 경계로

고유·검사 제약, 조건부 갱신과 행 잠금으로 최종 불변식 보호

불변식별 잠금

Redis 잠금을 전역 잠금이 아닌 경합 완화 수단으로 사용

대표 코드 근거

```
SqlCouponCampaignRepository.issueWithCapacityGuard
```

파일 backend/src/main/java/com/example/consistency/coupon/SqlCouponCampaignRepository.java

```
where id = ?
and status = 'ACTIVE'
and issued_count < capacity
for update
```

보호 규칙캠페인 행 잠금과 용량 조건을 한 SQL 흐름에서 판정해 동시 요청의 초과 발급을 차단합니다.

연결 테스트 backend/src/test/java/com/example/consistency/coupon/SqlCouponCampaignRepositoryTest.java
SqlCouponCampaignRepositoryTest.issueWithCapacityGuardUsesSingleStatementWithCampaignRowLock

검증 결과

- FirstLoginRewardDbConcurrency 동시 요청 8건 중 1건 지급·7건 거절을 assertion으로 고정합니다.
- PointSpendDbConcurrencyIT는 잔 100에서 60 차감 2건 중 1건만 성공해 잔액 40을 유지합니다.

구현 결과

동시 보상 요청 8건은 1건만 반영됐고, 60포인트 차감 2건은 잔액 40에서 수렴했으며 정원 3건을 초과한 쿠폰은 발급되지 않았습니다.

확인한 범위와 남은 경계

PostgreSQL·Redis·RabbitMQ 통합 경로를 Testcontainers 시나리오로 고정했으며 운영 규모 부하와 장시간 장애 복구 SLO는 포함하지 않았습니다.

Enterprise Policy RAG

AI 백엔드 / RAG / 권한 검색 / 운영 관측

담당 범위

2026.05-2026.06 /
개인 프로젝트 / AI
백엔드, 운영 콘솔, 검증
문서화

주요 기술

Python API

React/Vite

PostgreSQL/pgvector

외부 모델 선택 경로

비인가 검색과 근거 없는 생성을 답변 전에 차단하는 정책 RAG

권한 밖 문서를 검색·인용에서 제외하고 허용된 근거가 없으면 답변을 생성하지 않도록, 검색 전 권한 필터와 인용 검사를 적용했습니다.

시스템 흐름

실패 조건과 보호 경계



권한 검색
문서 5건 → 허용 3건
워크스페이스·소유자·부서 권한 선필터

답변 거절
근거 없음 → 답변 생성 안 함
빈 인용과 거절 사유 반환

인용 범위 검사
권한 밖 인용 0건
서비스 계층에서 접근 범위 재 검사

해결한 문제

비인가 문서 노출과 근거 없는 답변 생성

기업 내부 정책 RAG는 검색 품질만으로 부족합니다. 사용자가 볼 수 없는 문서가 검색 후보에 섞이지 않아야 하고, 답변에는 근거가 남아야 하며, 근거가 부족한 경우 거절 응답을 반환해야 합니다.

핵심 설계 판단

권한 검사 우선

답변 생성 전에 접근 가능한 문단만 검색 후보로 제한

근거 기반 거절

인용 근거가 없으면 답변과 인용을 비운 거절 응답을 반환

대표 코드 근거

RetrievalService.retrieve

파일 app/retrieval.py

```
access_reason = _access_reason(chunk, query)
if access_reason is None:
    continue
```

보호 규칙 저장소 필터 뒤에도 워크스페이스·소유자·부서 가시성을 다시 판정해 비인가 결과를 제거합니다.

연결 테스트 tests/test_retrieval_permissions.py
test_retrieval_allows_public_owner_and_department_documents_only

검증 결과

- 권한 검색 테스트에서 문서 5개 중 허용 범위 3개만 반환하고 다른 workspace 결과는 0건으로 고정합니다.
- 근거가 없으면 answer=null, citations=[]와 insufficient_evidence 사유를 반환합니다.

구현 결과

고정 문서 5건 중 권한이 허용된 3건만 검색하고, 허용된 근거가 없으면 답변과 인용을 비운 거절 응답을 반환했습니다.

확인한 범위와 남은 경계

기본 검증은 고정 입력과 메모리 저장소에서 수행했으며 PostgreSQL/pgvector·외부 모델 연동과 운영 인증은 선택 실행 범위입니다.

AI Gateway

LLM Gateway / 라우팅 /
비용 보호 / 캐시 / 폴백 /
관측

담당 범위

2026.06 / 개인 프로젝트
/ AI 백엔드·LLM 플랫폼
설계, 구현, 검증

주요 기술

Java 21

Spring Boot
WebFlux

Redis

PostgreSQL/pgvec
tor

예산 초과와 외부 모델 실패를 호출 전에 통제하는 LLM 게이트웨이

서비스마다 흩어진 LLM 인증·예산·캐시·라우팅·복구 정책을 OpenAI 호환 WebFlux 게이트웨이의 한
요청 처리 순서로 통합했습니다.



해결한 문제

애플리케이션마다 중복되는 LLM 정책과 외부 모델 장애 처리

LLM 기능을 여러 애플리케이션에 붙이면 외부 모델 SDK, 장애 대응, 토큰·비용 예산, 캐시, 보호 규칙과 요청 기록이 서비스마다 따로 구현됩니다. 게이트웨이가 공통 정책과 요청 기록을 처리하도록 구성했습니다.

핵심 설계 판단

공통 호환 API

OpenAI 호환 /v1/chat/completions로
클라이언트 변경 최소화

고정된 정책 순서

인증·할당량·입출력 검사·캐시·라우팅·복구·기록
순서 고정

대표 코드 근거

GatewayPipeline.execute

파일 backend/src/main/java/com/example/gateway/api/GatewayPipeline.java

```
QuotaOutcome quota = quotaGuard.evaluate(request,
estimatedTokens);
if (quota != QuotaOutcome.ALLOWED) {
    return rejected(request, mode, quota,
elapsed(start));
}
```

보호 규칙 예산 토큰으로 사용자 조직의 할당량을 먼저 검사해 초과 요청을
외부 모델 호출 전에 종료합니다.

연결 테스트 backend/src/test/java/com/example/gateway/observability/RequestLogStoreTest.java
RequestLogStoreTest.quotaRejectPathCreatesRowWithNullProviderAndOutcome

검증 결과

- ChatCompletionController 테스트가 JSON 일괄 응답과 SSE 스트리밍 경로를 분리해 검증합니다.
- 인증 누락 401, 잘못된 요청 400과 할당량·예산 보호 규칙을 API 테스트로 고정합니다.

구현 결과

재시도 예산이 소진되면 다음 외부 모델 호출을 차단하고, 할당량 초과 요청은 모델을 호출하지 않은 채 거절하도록 검증했습니다.

확정한 범위와 남은 경계

기본 검증은 고정 응답 외부 모델과 메모리 저장소를 사용했으며 실제 모델 전송·운영 저장소·클라우드 배포는 포함하지 않았습니다.

CDC Data Platform

CDC / Kafka /
Lakehouse / Lineage /
Replay

담당 범위

2026.06 / 개인 프로젝트
/ CDC 데이터 플랫폼
백엔드 설계, 구현, 검증

주요 기술

Spring Boot

PostgreSQL

Debezium

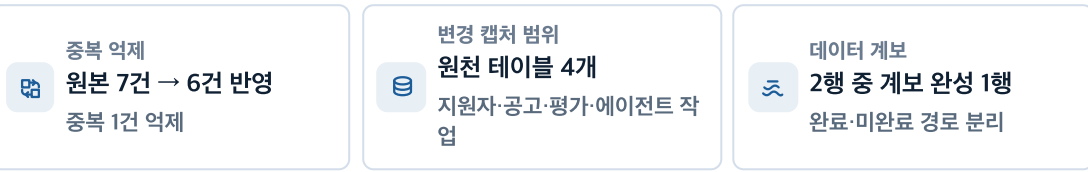
Kafka

변경 캡처·복구·저장을 독립 경계에서 검증한 CDC 플랫폼

PostgreSQL 변경 캡처, 표준 이벤트 제어와 레이크하우스 저장을 세 독립 실험으로 구현했습니다. 각 경계에서 원천 메타데이터, 중복 처리 장부, 재처리 상태와 데이터 계보를 검증했습니다.

시스템 흐름

실패 조건과 보호 경계



해결한 문제

중복 CDC, 복구 이력 손실과 자동 연결되지 않은 구간의 중단 전달

CDC 이벤트는 커넥터 재시작과 재전달로 중복될 수 있고 적재 실패 뒤 복구 이력이 사라질 수 있습니다. 수집·제어·적재 구간이 자동 연결되지 않아 중단 전달과 재처리를 보장할 수 없는 범위도 명시했습니다.

핵심 설계 판단

원천 메타데이터 기반 이벤트 ID

LSN·오프셋을 이벤트 ID와 재처리·계보 추적 기준으로 사용

복구 상태 분리

원본·표준 이벤트·재시도·DLQ·재처리 상태를 분리해 원인과 복구를 추적

대표 코드 근거

CanonicalIngestService.ingestRawEnvelope

파일 backend/src/main/java/com/example/cdcplatform/event/CanonicalIngestService.java

```
DebeziumEnvelope envelope =  
    DebeziumEnvelope.parse(rawEnvelopeJson);  
CanonicalCdcEvent event =  
    CanonicalCdcEvent.fromEnvelope(envelope);  
String sourceOffsetJson =  
    sourceOffsetJson(envelope);  
boolean created =  
    canonicalEventPublisher.recordCanonicalEvent(event,  
        sourceOffsetJson);
```

보호 규칙 Debezium 원본을 표준 이벤트로 정규화하면서 원천 오프셋을 보존하고 처리 장부 결과로 중복을 판정합니다.

연결 테스트 backend/src/test/java/com/example/cdcplatform/event/CanonicalIngestServiceTest.java
CanonicalIngestServiceTest.duplicateRawEnvelopeDoesNotIncreaseCanonicalEventCount

검증 결과

- 로컬 계보 고정 입력이 원본 이벤트 7건 중 6건을 반영하고 중복 1건을 억제합니다.
- 지원자·공고·평가·에이전트 작업 원천 테이블 4개의 메타데이터와 계보 키를 보존합니다.

구현 결과

원본 이벤트 7건 중 6건을 반영하고 중복 1건을 억제했으며, 4개 원천 테이블과 분석 마트 사이의 계보 식별자를 보존했습니다.

확인한 범위와 남은 경계

CDC 실행 환경, 제어 API와 레이크하우스는 독립 검증 상태이며 중단 연결은 구현하지 않았습니다. Kafka 토픽 상시 소비, 자동 적재와 AWS S3/Athena·Trino 실행은 포함하지 않았습니다.

Fashion Personalization Platform

Python API / 커머스
개인화 / 추천 배치 /
이벤트 처리

담당 범위

2026.07 / 개인 프로젝트
/ Python 백엔드, 이벤트
처리, 추천 배치, 운영
전환 설계

주요 기술

Python 3.11+

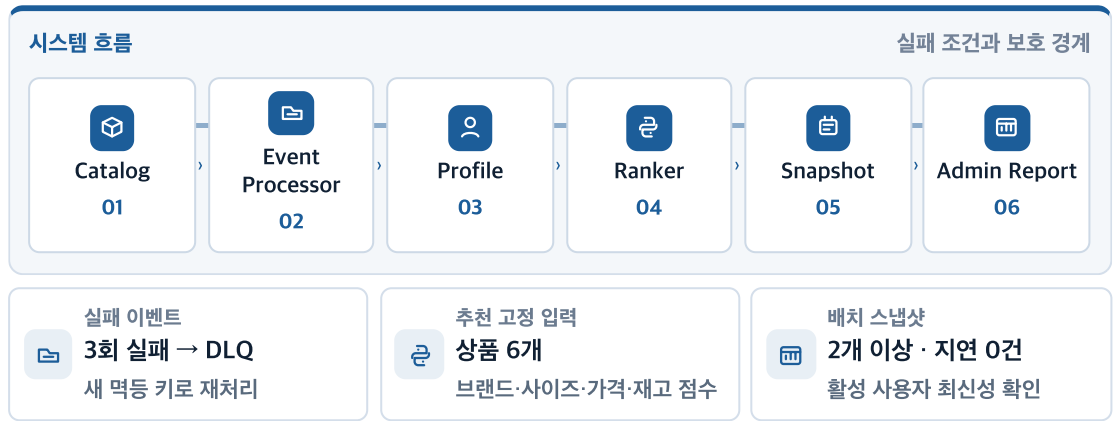
선택형 FastAPI

pytest

이벤트 처리

실패 이벤트를 격리하고 추천 최신성을 추적하는 개인화 백엔드

중복·실패 행동 이벤트가 사용자 프로필을 오염시키지 않도록 먹등 처리와 DLQ를 적용하고, 추천 점수와 스냅샷 최신성을 함께 기록했습니다.



해결한 문제

중복·실패 이벤트가 프로필과 추천 스냅샷을 오염함

패션 커머스 추천은 브랜드, 카테고리, 태그, 사이즈, 가격대와 재고 같은 상품 정보에 사용자 행동 이벤트를 함께 반영해야 합니다. 중복 이벤트나 처리 실패가 프로필을 오염시키지 않도록 이벤트 처리, 배치 스냅샷과 관리자 리포트를 분리했습니다.

핵심 설계 판단

외부 의존성 없는 핵심 로직

외부 설치 없이 도메인·서비스 흐름을 검증

실패 이벤트 격리

먹등 키·재시도 예산·DLQ·재처리로 오염 차단

대표 코드 근거

apply_event_to_profile

파일 src/fashion_personalization/recommendation.py

```
selected_size = event.payload.get("size")
if isinstance(selected_size, str) and
selected_size in product.sizes:
    _add_weight(profile.size_preferences,
selected_size, max(weight, 0.1))
```

보호 규칙선택한 값이 실제 상품 옵션에 있을 때만 사이즈 선호에 반영해 잘못된 프로필 신호를 막습니다.

연결 테스트 tests/test_recommendation_engine.py
test_size_preference_contributes_to_ranking

검증 결과

- EventProcessor 테스트가 동일 idempotency key를 한 번만 반영하고 3회 실패 뒤 DLQ로 격리합니다.
- 추천 테스트가 6개 상품 fixture에서 브랜드·사이즈·가격·재고 조건을 deterministic ranking에 반영합니다.

구현 결과

동일 이벤트 키는 한 번만 반영되고 3회 실패한 이벤트는 DLQ로 격리됐으며, 활성 사용자 스냅샷 2개 이상에서 지연 0건을 확인했습니다.

확인한 범위와 남은 경계

검증 구현은 메모리 저장소를 사용하며 FastAPI는 선택 어댑터입니다.
PostgreSQL·SQS/EventBridge·S3·AWS 배포는 포함하지 않았습니다.

01 StockRush

확인 결과

동일 이벤트 2회에도 Outbox 1건, 늦은 결제 승인 무시, 발행 5회 실패 뒤 FAILED

아직 확인하지 않은 범위

실결제 연동, 다중 리전, 운영 규모 부하와 발행기 고가용성

03 Member Event Consistency

확인 결과

동시 8건→보상 1건, 60원 차감 2건→잔액 40, 용량 3→정확히 3건 발급

아직 확인하지 않은 범위

운영 규모 부하, 장시간 Redis/RabbitMQ 장애 복구와 실서비스 SLO

02 Enterprise Policy RAG

확인 결과

문서 5건 중 허용 3건만 검색, 근거가 없으면 답변과 인용을 비운 거절 응답

아직 확인하지 않은 범위

운영 OIDC, 관리형 PostgreSQL/pgvector, 실제 OpenAI 호출과 배포 스모크 테스트

04 AI Gateway

확인 결과

8단계 처리 순서, 4개 실행 방식과 4개 라우팅 전략을 회귀 테스트로 고정

아직 확인하지 않은 범위

실제 OpenAI/Anthropic 전송, 운영 Redis/PostgreSQL, 클라우드 배포와 부하

05 CDC Data Platform

확인 결과

원본 7건 중 6건 반영·중복 1건 억제, 4개 원천 테이블과 분석 마트의 계보 추적

아직 확인하지 않은 범위

AWS S3/Athena, 장기 Kafka Connect, Trino, 운영 규모 부하와 다중 리전

06 Fashion Personalization Platform

확인 결과

동일 키 1회 반영, 3회 실패 뒤 DLQ, 상품 6개 추천 순위와 스냅샷 2개 이상·지연 0건

아직 확인하지 않은 범위

PostgreSQL·SQS/EventBridge·S3 adapter, AWS 배포, 실제 ML·A/B 성과